

III Graph-Based Logistics Project Optimizer (GLPO)

Tài Liệu Định Hướng & Thiết Kế Kiến Trúc Hệ Thống (Ultimate Guide)

Chào mừng bạn đến với tài liệu định hướng tổng thể dự án **GLPO**. Tài liệu này đóng vai trò là kim chỉ nam giúp bạn hiểu rõ lộ trình thực hiện đồ án, các đầu việc cần làm của từng pha, và các mô hình đồ thị/AI sẽ được áp dụng xuyên suốt hệ thống.

MỤC LỤC

- [Định Hướng Đề Tài](#)
- [Lộ Trình Triển Khai & Đầu Việc Chi Tiết \(5 Phases\)](#)
- [Các Mô Hình Đồ Thị & AI Áp Dụng \(Graph-Based Models\)](#)
- [Ứng Dụng Dữ Liệu Case Study MFET 3008/5040](#)
- [Lựa Chọn Công Nghệ & Các Quyết Định Đánh Đổi \(Trade-offs\)](#)

1. Định Hướng Đề Tài

Dự án này là một **Hệ thống hỗ trợ ra quyết định thông minh (Smart Decision Support System)** dành cho quản lý dự án logistics. Thay vì quản lý công việc thủ công, hệ thống mô hình hóa dự án thành một **Đồ thị có hướng không chu trình (DAG)** $G = (V, E)$.

Hệ thống hướng tới việc kết hợp 3 trụ cột công nghệ chính:

- Lý thuyết Đồ thị thuần túy:** Phân tích cấu trúc mạng công việc (CPM, Topological Sort, Slack).
- Machine Learning đồ thị:** Dự đoán trước sự biến động tiến độ do tác động ngoại cảnh (thời tiết, nhà cung cấp).
- Toán tối ưu lai (Hybrid Optimization):** Tự động giải quyết xung đột tài nguyên và tìm phương án crash tiến độ với chi phí rẻ nhất.

2. Lộ Trình Triển Khai & Đầu Việc Chi Tiết (5 Phases)

Dưới đây là bảng phân chia các đầu việc cụ thể cần thực hiện qua từng giai đoạn phát triển:

Pha 1: Core Graph DAG Engine (Xây dựng động cơ đồ thị ở Backend)

Tập trung thiết lập nền tảng toán học đồ thị thuần túy bằng Python.

- [] Đầu việc 1.1:** Định nghĩa cấu trúc dữ liệu cho Đỉnh (Task node: Lưu tên, thời gian dự kiến, tài nguyên yêu cầu) và Cạnh (Dependency directed edge: Quan hệ phụ thuộc).

- [] **Đầu việc 1.2:** Viết thuật toán phát hiện chu trình (Cycle Detection) bằng DFS để đảm bảo đồ thị luôn hợp lệ (DAG), không bị lặp vô hạn.
- [] **Đầu việc 1.3:** Viết thuật toán Sắp xếp Topo (Topological Sort) để lập chuỗi thứ tự thực thi công việc khoa học.
- [] **Đầu việc 1.4:** Lập trình giải thuật **CPM (Critical Path Method)**:
 - *Duyệt xuôi (Forward Pass):* Tính Early Start (ES), Early Finish (EF).
 - *Duyệt ngược (Backward Pass):* Tính Late Start (LS), Late Finish (LF).
 - *Tính toán thời gian dự trữ:* Total Slack (TS) và Free Slack (FS) của từng nút.
- [] **Đầu việc 1.5:** Viết thuật toán xác định các nút găng (Slack = 0) và nối chúng thành **Đường găng (Critical Path)**.

Pha 2: Cấu Trúc Cơ Sở Dữ Liệu & API (Database & API Setup)

Thiết lập lớp lưu trữ dữ liệu và cổng kết nối API.

- [] **Đầu việc 2.1:** Thiết kế cơ sở dữ liệu PostgreSQL gồm các bảng: **Projects**, **Tasks**, **Dependencies (Edges)**, **Resources** (Tài nguyên nội bộ/thầu phụ) và **EVM_Checkpoints**.
- [] **Đầu việc 2.2:** Viết các câu lệnh truy vấn đệ quy **CTE (Common Table Expressions)** trong PostgreSQL để duyệt cây công việc phụ thuộc một cách nhanh nhất.
- [] **Đầu việc 2.3:** Thiết lập FastAPI, viết các API endpoints: API CRUD dự án/công việc, API phân tích CPM trả về kết quả dạng JSON.

Pha 3: Bộ Tối Ưu Hóa Lai (Hybrid Project Optimizer - MILP + GA)

Xây dựng bộ não tối ưu hóa tiến độ và chi phí.

- [] **Đầu việc 3.1:** Lập trình mô hình toán học Quy hoạch tuyến tính nguyên hỗn hợp (MILP) bằng thư viện PuLP để giải bài toán Crashing tuyến tính (overtime, contractor) cho các đồ thị dự án nhỏ.
- [] **Đầu việc 3.2:** Lập trình thuật toán di truyền (GA) bằng thư viện DEAP để xếp lịch khi tài nguyên nhân sự bị giới hạn chéo nhau (bài toán RCPSP).
- [] **Đầu việc 3.3:** Viết cơ chế tối ưu hóa lai: Nghiệm tối ưu từ MILP sẽ được đưa vào làm hạt giống (seed) khởi tạo quần thể cho GA để tăng tốc độ hội tụ của GA trên đồ thị lớn.

Pha 4: Giao Diện Trực Quan Đồ Thị Tương Tác (Frontend UI)

Thiết kế giao diện hiện đại để người dùng tương tác trực quan với đồ thị.

- [] **Đầu việc 4.1:** Thiết lập dự án Frontend React/Next.js kết hợp TailwindCSS.
- [] **Đầu việc 4.2:** Tích hợp thư viện **React-Flow** làm Graph Editor: Hỗ trợ người dùng kéo thả các node công việc và nối mũi tên phụ thuộc trực tiếp.
- [] **Đầu việc 4.3:** Tích hợp thư viện **Cytoscape.js** (chạy Dagre layout) để tự động sắp xếp sơ đồ mạng lưới hoạt động AON và tô đỏ nổi bật các nút nằm trên Đường găng.

- [] **Đầu việc 4.4:** Thiết lập Gantt Chart tương tác hiển thị tiến độ thời gian thực đồng bộ với đồ thị.

Pha 5: AI Predictive Module & EVM Dashboard (AI & Kiểm Soát)

Tích hợp trí tuệ nhân tạo và kiểm soát tiến độ thực tế.

- [] **Đầu việc 5.1:** Trích xuất các đặc trưng topo của đồ thị từ Backend (In/Out-degree, Centrality...) làm dữ liệu huấn luyện ML.
- [] **Đầu việc 5.2:** Huấn luyện mô hình Học máy để dự đoán biến động thời gian thực tế của công việc dựa trên dữ liệu môi trường (thời tiết, hiệu suất nhân lực).
- [] **Đầu việc 5.3:** Xây dựng màn hình EVM Dashboard: Vẽ biểu đồ PV, EV, AC và hiển thị chỉ số CPI, SPI để cảnh báo rủi ro trễ hạn.

□ 3. Các Mô Hình Đồ Thị & AI Áp Dụng (Graph-Based Models)

Để tối ưu hóa dự án, bạn sẽ áp dụng các mô hình học thuật đồ thị sau:

1. DAGNN (Directed Acyclic Graph Neural Networks):

- **Mục đích:** Dự đoán thời gian thực tế (\$d_i\$) của công việc.
- **Tại sao:** GCN thông thường không hiểu thứ tự dòng chảy (partial ordering). DAGNN lan truyền thông tin theo chiều có hướng của các cạnh, giúp dự báo thời gian thực tế của một nút dựa trên "áp lực" tích lũy từ các nút tiền quyết của nó.

2. GAT (Graph Attention Networks):

- **Mục đích:** Dự báo rủi ro lan truyền trễ tiến độ (Delay Propagation).
- **Tại sao:** GAT gán trọng số "attention" khác nhau cho các cạnh liên kết. Các nút nằm trên đường găng (Critical Path) sẽ có mức độ chú ý cao hơn, từ đó cảnh báo chính xác rủi ro khi có sự chậm trễ xảy ra.

3. GNN + Deep Reinforcement Learning (DRL) / Hybrid Solver:

- **Mục đích:** Giải bài toán xếp lịch giới hạn tài nguyên (RCPSP).
- **Tại sao:** Trạng thái đồ thị (DAG state) được nhúng qua GNN thành vector, RL Agent dựa vào đó để quyết định chọn Task nào tiếp theo để xếp lịch nhằm tối thiểu hóa tổng thời gian (Makespan). Hoặc kết hợp toán học MILP và GA để giải tối ưu đa mục tiêu.

□ 4. Ứng Dụng Dữ Liệu Case Study MFET 3008/5040

Bộ tài liệu Case Study của bạn là nguồn dữ liệu kiểm thử (Test Case) hoàn hảo để xác minh tính chính xác của các thuật toán trên đồ thị:

- **Đầu vào:** Đồ thị DAG gồm 21 hoạt động (A → U) và ràng buộc về 8 loại nhân sự kỹ sư.
- **Bài toán Tối ưu hóa:** Tìm phương án thuê thầu phụ (Contractor) và làm thêm giờ (Saturday 1.5x, Sunday 3.0x) sao cho tổng chi phí là nhỏ nhất dựa trên biểu đồ tài nguyên chéo nhau.

- **Ground Truth (Kết quả gốc đối chứng):**
 - Thời gian dự án bình thường: **66 ngày**.
 - Đường găng: **$A \rightarrow C \rightarrow G \rightarrow I \rightarrow J \rightarrow N \rightarrow R \rightarrow T \rightarrow U$** .
 - Thời gian dự án khi bị giới hạn tài nguyên: **96 ngày**.
 - Chi phí tối ưu khi thuê thầu phụ: **\$70,230**.
-

⚖️ 5. Lựa Chọn Công Nghệ & Các Quyết Định Đánh Đổi (Trade-offs)

Quyết định 1: Giao diện Đồ thị tương tác (Frontend)

- **Lựa chọn:** Kết hợp React-Flow và Cytoscape.js.
- **Đánh đổi:** React-Flow đem lại trải nghiệm kéo thả mượt mà ở chế độ chỉnh sửa đồ thị, trong khi Cytoscape.js giải quyết xuất sắc bài toán tự động sắp xếp (auto-layout) sơ đồ AON phân cấp phức tạp.

Quyết định 2: Lưu trữ cấu trúc Đồ thị (Database)

- **Lựa chọn:** PostgreSQL kết hợp Recursive CTE.
- **Đánh đổi:** Tránh được sự chồng chéo và khó tích hợp của các Graph Database (như Neo4j), trong khi vẫn duyệt cây phụ thuộc cực kỳ nhanh chóng và quản lý các thuộc tính tài nguyên (chi phí, nhân sự) dễ dàng thông qua các bảng quan hệ.

Quyết định 3: Bộ tối ưu hóa (Optimization Engine)

- **Lựa chọn:** Bộ tối ưu hóa Lai (Hybrid MILP + Genetic Algorithm).
 - **Đánh đổi:** MILP giải quyết chính xác 100% (Exact Solution) các bài toán quy mô vừa/nhỏ trong thời gian mili-giây. GA giải quyết các bài toán lớn hơn hoặc phi tuyến tính. Sự kết hợp lai (lấy nghiệm MILP làm hạt giống cho GA) giúp GA hội tụ nhanh hơn gấp nhiều lần.
-

📦 6. Các Mô Hình Tham Khảo Trên Hugging Face (Hugging Face Models)

Để phục vụ lập trình và làm giàu cơ sở tài liệu cho đồ án, dưới đây là các mô hình và tập dữ liệu uy tín trên Hugging Face có thể tham khảo hoặc kế thừa:

1. Mô hình **ICICLE-AI/FoodFlow_GNN_Model** (Đồ thị & Dự báo dòng chảy):

- **Thể loại:** Graph Convolutional Networks (GCN) + Graph Attention Networks (GAT).
- **Ứng dụng:** Dự báo luồng và lưu lượng logistics thực tế giữa các vùng. Hữu ích để tham khảo cách thức GAT gán trọng số chú ý cho các cạnh liên kết.

2. Mô hình **keras-io/graph-attention-nets** (Graph Attention Networks):

- *Thể loại:* GAT (TensorFlow/Keras).
- *Ứng dụng:* Mã nguồn mẫu chuẩn hóa để học biểu diễn node (Node Representation Learning) trên đồ thị phục vụ phân loại và dự báo thuộc tính.

3. Mô hình HUANG1993/GreedRL -VRP - pretrained -v1 (Học tăng cường cho Định tuyến):

- *Thể loại:* Pre-trained Deep Reinforcement Learning (GreedRL).
- *Ứng dụng:* Giải bài toán tối ưu tổ hợp VRP/TSP trên đồ thị, học cách đưa ra các quyết định dịch chuyển tối ưu giữa các đỉnh.

4. Dataset Yahias21/vrp_benchmark (Dữ liệu thử nghiệm):

- *Thể loại:* Benchmark cho Stochastic Vehicle Routing Problem (SVRP).
- *Ứng dụng:* Cung cấp các kịch bản đồ thị vận chuyển thực tế từ 10 đến 1000 khách hàng, có tích hợp các yếu tố ngẫu nhiên như **trễ thời gian do thời tiết/giao thông** để huấn luyện mô hình dự báo trễ tiến độ.